Non Sequitur Publishing · White Paper

Agentic Substrate: What an AI System Must Provide to Execute Autonomous Tasks Under Governance Constraints

Justin H. Kuiper, CISSP

Written: 2026-04-25 · v1.0-preprint

Abstract

The peer-reviewed multi-agent literature has demonstrated that agentic systems can reason, coordinate, and use tools at a level sufficient for many production task classes. AutoGen, MetaGPT, ChatDev, ReAct, Reflexion, and Toolformer between them describe agent architectures whose capability is no longer the bottleneck for autonomous task execution. The bottleneck is governability: capability alone does not produce execution that humans can authorize, audit, and refuse. Whether governance is even expressible in a given deployment depends not on the protocol the team specifies, but on the substrate the protocol must run over.

This paper specifies the agentic substrate as a governance surface. It treats the substrate as a designed layer with named primitives — agent anatomy (memory, reasoning, tool access), orchestration topology (single-agent, multi-agent, swarm), governance integration (admission hooks, evidence channels, manifest propagation, HITL escalation), and runtime isolation (process boundaries, capability tokens, sandboxed tools, scoped credentials) — each carrying explicit governance contracts. The contracts are substrate-independent: any compliant implementation must honor them regardless of whether the underlying composition runs on LangGraph, AutoGen, CrewAI, the Model Context Protocol, the OpenAI Agents SDK, or a bespoke stack.

Four claims structure the paper. Agent anatomy is a governance surface, not a capability surface: memory, reasoning, and tools each carry governance obligations that determine whether the agent is governable at all. Orchestration topology determines authority distribution: single-agent, multi-agent, and swarm orchestrations each impose a different authority structure on the Governance Layer / Orchestrator / Executor split, and substrate selection determines which is implementable. Governance integration is substrate-layer, not protocol-layer: protocol specifies what the gates do; the substrate specifies how they are invoked, and only substrate-side enforcement is structural. Runtime isolation is the blast-radius contract: process isolation, capability tokens, sandboxes, and credential scoping define the upper bound on harm an admission failure can produce.

Paper Seven is the seventh paper in the Mission-Ready AI decalogy and the second of three Application-layer depth papers. It operationalizes Skipjack §6 ("What Skipjack Requires of the Harness Substrate") and Paper Six §6.1 (the five operating-model contracts) into a substrate specification.

How to cite

Justin H. Kuiper, CISSP (2026). *Agentic Substrate: What an AI System Must Provide to Execute Autonomous Tasks Under Governance Constraints* (v1.0-preprint). Non Sequitur Publishing.
https://nonsequitur.tech/white-papers/agentic-substrate/

ABSTRACT

The peer-reviewed multi-agent literature has demonstrated that agentic systems can reason, coordinate, and use tools at a level sufficient for many production task classes. AutoGen, MetaGPT, ChatDev, ReAct, Reflexion, and Toolformer between them describe agent architectures whose capability is no longer the bottleneck for autonomous task execution. The bottleneck is governability: capability alone does not produce execution that humans can authorize, audit, and refuse. Whether governance is even expressible in a given deployment depends not on the protocol the team specifies, but on the substrate the protocol must run over. A substrate that does not surface admission as a refusal point cannot honor the Skipjack admission gate. A substrate that does not partition cognitive modes structurally cannot honor the multi-account discipline of the operating model. A substrate that does not propagate the context integrity manifest across handoffs cannot honor the Framework's C^3 contract.

This paper specifies the agentic substrate as a governance surface. It treats the substrate as a designed layer with named primitives — agent anatomy (memory, reasoning, tool access), orchestration topology (single-agent, multi-agent, swarm), governance integration (admission hooks, evidence channels, manifest propagation, HITL escalation), and runtime isolation (process boundaries, capability tokens, sandboxed tools, scoped credentials) — each carrying explicit governance contracts. The contracts are substrate-independent: any compliant implementation must honor them regardless of whether the underlying composition runs on LangGraph, AutoGen, CrewAI, the Model Context Protocol, the OpenAI Agents SDK, or a bespoke stack.

Four claims structure the paper. *Agent anatomy is a governance surface, not a capability surface*: memory, reasoning, and tools each carry governance obligations that determine whether the agent is governable at all. *Orchestration topology determines authority distribution*: single-agent, multi-agent, and swarm orchestrations each impose a different authority structure on the Governance Layer / Orchestrator / Executor split, and substrate selection determines which is implementable. *Governance integration is substrate-layer, not protocol-layer*: protocol specifies what the gates do; the substrate specifies how they are invoked, and only substrate-side enforcement is structural. *Runtime isolation is the blast-radius contract*: process isolation, capability tokens, sandboxes, and credential scoping define the upper bound on harm an admission failure can produce.

Paper Seven is the seventh paper in the Mission-Ready AI decalogy and the second of three Application-layer depth papers. It operationalizes Skipjack §6 ("What Skipjack Requires of the Harness Substrate") and Paper Six §6.1 (the five operating-model contracts) into a substrate specification.

---

1. THE SUBSTRATE PROBLEM: WHY AGENT ARCHITECTURES NEED A SHARED FOUNDATION

1.1 CAPABILITY HAS BEEN DEMONSTRATED; GOVERNABILITY HAS NOT BEEN SUBSTRATE-GROUNDED

The peer-reviewed agent literature has, in the last three years, supplied much of what was missing from the early generation of language-model-based systems. AutoGen[1] specified a multi-agent conversational framework in which agents with named roles negotiate task decomposition through structured exchange. MetaGPT[2] adopted standardized operating procedures from human software-engineering practice and showed that agent teams could follow them. ChatDev[3] demonstrated waterfall-style decomposition across CEO / CTO / programmer / reviewer agents on small software projects. ReAct[4] formalized the reason-then-act loop that has become the structural primitive for tool-using agents. Reflexion[5] added self-correction over multiple reasoning episodes. Toolformer[6] showed that language models could be trained to invoke external tools in the middle of generation. The capability story is rich and improving.

The governance story is not. The four most-deployed production harness frameworks — LangChain/LangGraph, CrewAI, the Model Context Protocol, and the OpenAI Agents SDK — have, as of this writing, no peer-reviewed publication.[7] Governance design, accountability structure, and the substrate-side primitives that make governance enforceable receive minimal

---

1 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” arXiv preprint arXiv:2308.08155 (2024), https://arxiv.org/abs/2308.08155.

2 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, https://arxiv.org/abs/2308.00352.

3 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, “ChatDev: Communicative Agents for Software Development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, https://arxiv.org/abs/2307.07924.

4 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023, https://arxiv.org/abs/2210.03629.

5 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2303.11366.

6 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2302.04761.

treatment in their documentation, partly because they are difficult problems and partly because the production-deployment audience for the harnesses has not historically demanded it.

The asymmetry between capability and governability is a structural observation about where the research investment has gone. Multi-agent capability research is a tractable academic program with publishable results: papers can demonstrate task completion, coordination success, and reasoning-loop convergence on benchmarks. Substrate-level governance research is harder to publish: the result is typically a contract specification or a refused-execution log rather than a benchmark improvement. It is not surprising that the research record looks the way it does. The consequence is that production deployment of governance-grade agentic systems operates on a substrate with weak academic grounding for the very properties production deployment most needs.

The decalogy fills the gap by working downward from Framework (HGC³AE²) through Protocol (Skipjack) to Application (the operating model and the substrate that carries it). The substrate's role in that flow is specific: it is the layer at which the doctrine encoded above becomes physically expressible or fails to. Skipjack §6 already named what the protocol requires of the substrate.[8] This paper specifies the substrate side of those requirements: not what the protocol expects, but what a compliant substrate must provide.

1.2 THE TWO FAILURE MODES OF SUBSTRATE SELECTION WITHOUT GOVERNANCE GROUNDING

Two failure modes follow predictably from substrate selection that does not begin with governance fitness. Both are observable in early agentic adoption and both are diagnosable from the resulting deployment, often after a shipped failure has surfaced the gap.

The first is *capability-first selection*. The team picks the harness with the best reasoning, coordination, or tool-use story. The selection criteria are throughput, benchmark scores, ecosystem maturity, and developer ergonomics. Governance is acknowledged as important but is treated as something to add on top — through prompt engineering, through wrapper code, through manual review. The harness then fails, in operation, at admission: tasks reach agents without scope manifests, scope-deviating outputs are produced quickly, and the team discovers, by reviewing what was actually shipped, that several agents acted outside the envelopes the team had thought it was enforcing. The team then rebuilds governance on top of capability that does not natively expose the right primitives. The rebuild is expensive and partial, because the substrate does not provide the hooks the rebuild requires; the team writes orchestrator code that audits agent outputs after the fact rather than gating them before execution, because the substrate did not surface the pre-execution interception point that admission gating requires. AE² fails open. The cycle's outputs are not auditable back to a Governance-Layer authorization, because no admission decision was recorded at the substrate boundary.

The second is *governance-first specification without substrate analysis*. The team writes a thoughtful governance specification, often based on HGC³AE² or a comparable framework, and assumes the substrate will accommodate it. The specification names admission gates, evidence channels, manifest propagation, and HITL control points. The team chooses a harness —

---

7 As of the writing of this paper, none of the four most-deployed production agent harness frameworks — LangChain/LangGraph, CrewAI, the Model Context Protocol, or the OpenAI Agents SDK — has produced a peer-reviewed publication. This observation should be updated if peer-reviewed publications appear for these frameworks.

8 Kuiper, "Skipjack Protocol," §6 (Skipjack and the Production Harness Layer), particularly §6.4 (What Skipjack Requires of the Harness Substrate).

perhaps one of the peer-reviewed frameworks above — and begins implementation. The substrate accepts the high-level shape of the specification but does not enforce it: the admission hook can be bypassed by an agent that addresses the underlying tool API directly, the evidence channel is one log stream among many that the orchestrator must remember to consult, the manifest exists as a Python object that any agent can mutate. The governance becomes nominal rather than structural. A capable agent that can route around a soft gate eventually does so, not because the agent is adversarial but because reasoning loops will, over enough executions, find paths that a reviewing human would not have authorized. The team has the right specification and the wrong substrate.

Both failure modes share a structural cause: substrate selection is treated as a capability decision rather than a governance decision. Capability matters; benchmark scores matter; ecosystem maturity matters. But these factors enter the selection process after governance fitness has been assessed, not before. A harness that is capable but cannot enforce admission discipline is a poor governance substrate regardless of its benchmark scores.

The substrate this paper specifies is not a particular harness. It is a set of governance contracts that any substrate must satisfy for the protocol layer above to be enforceable. The substrate-selection process becomes: enumerate the contracts, assess each candidate's coverage, identify the gaps, choose either to compose around them or to engineer them. The choice is consequential. The contracts are not.

1.3 WHAT THE SUBSTRATE MUST SURFACE

Five surfaces appear repeatedly across the protocol layer (Skipjack §3–§6) and the operating-model layer (Paper 6 §6.1). They are the recurring shape of what the substrate must provide.

Admission as a refusal point. The orchestrator must be able to refuse a task before the executing agent receives it. Refusal must be hard: the substrate must not provide a path by which a refused task reaches the agent through an alternative route. A substrate that exposes admission as an advisory check that the agent can override is not enforcing admission; it is suggesting it.

Evidence as a first-class channel. Every executed task must emit structured evidence — input received, action executed, output produced, deviations from envelope, manifest version honored — through a substrate-side channel distinct from agent stdout, distinct from tool returns, distinct from the harness's diagnostic logging. The orchestrator audits the channel before declaring the task complete. A substrate that conflates evidence with diagnostic logging makes the governance audit a forensic exercise rather than a structural one.

Partition isolation as an invariant. Cognitive modes must have substrate-side partitions, not prompt-side personas. A persona-prompt on a shared underlying context does not satisfy the contract; the saturation mechanism described in Paper Two[9] still applies, and the contexts contaminate each other regardless of prompt-level instructions to the contrary. Partition isolation must be at the layer where context actually lives — process boundaries, distinct memory stores, namespace separation.

9 Justin H. Kuiper, CISSP, "Epistemic Constraints and Semantic Compression in Natural Language Processing: A Theoretical Foundation for the HGC³AE² Framework," Non Sequitur Publishing, 2026, v1.0-preprint, https://nonsequitur.tech/white-papers/epistemic-constraints/.

Manifest propagation across handoffs. The Skipjack context integrity manifest[10] must travel with the work, not with the operator's memory. Substrate-side serialization, transport, and deserialization preserve the manifest across every node in the execution chain. A substrate that requires the operator or the agent to re-attach the manifest at each handoff is one in which the manifest is a convention, not a contract.

Capability scoping per agent. An agent's tool registry is bounded by its admission envelope, not by the harness's global registry. A capability the envelope does not authorize is not invokable by the agent during that execution, full stop. Scoping must be substrate-side because agents that can introspect the global registry will, eventually, attempt to use capabilities outside the envelope when the reasoning loop proposes it as plausible.

These five surfaces are the substrate's job. The remainder of this paper specifies how a substrate provides them: through agent anatomy (§2), through orchestration topology (§3), through governance integration (§4), and through runtime isolation (§5). The substrate's scaling behavior across deployment tiers (§6) and the contracts that any harness implementation must honor (§6.1) close the substrate-specification arc.

1.4 WHAT THIS PAPER DOES NOT ADDRESS

The decalogy distinguishes among substrate, operating model, and deployment readiness. Paper Seven is the substrate paper. Three adjacent topics are explicitly out of scope.

The first is *operating-model cadence*. The standup analogue, sprint planning, completion review, and retrospective ceremonies — the day-to-day form of the Skipjack Protocol applied to a team-sized unit of work — are specified in Paper Six.[11] This paper assumes the operating model and specifies what the substrate must provide for the model to be implementable.

The second is *deployment readiness, observability runbooks, and decommissioning*. Once the operating model is running on a compliant substrate, deploying the result to production users with audit-grade evidence behind it requires pre-deployment audit, operations telemetry, incident response, and a documented decommissioning path. Paper Eight will address those concerns. This paper specifies what runs; Paper Eight will specify how the running thing reaches end users.

The third is *empirical substrate benchmarking*. Quantitative comparison of specific harnesses against specific governance contracts requires controlled experimentation that the capstone implementation paper (Paper Nine) and the capstone results paper (Paper Ten) will produce. The substrate-research dossier maintained by the substrate-research persona (yks-build #73 / yks-spine-binder #227) carries the working notes that feed those papers. This paper specifies the contracts; the empirical fitness is reported elsewhere.

1.5 POSITION IN THE STACK

This paper elaborates the agentic substrate at the Application layer of the governance flow the series describes. HGC³AE² (Paper 1) is the Framework layer that defines the doctrine.[12]

---

10 Kuiper, "Skipjack Protocol," §4.2 (The Context Integrity Manifest in Practice).

11 Justin H. Kuiper, CISSP, "Operating Model for Agentic Teams: Standup, Sprint Planning, and Context Discipline When the Dev Team Is Non-Human," Non Sequitur Publishing, 2026, v1.0-preprint, https://nonsequitur.tech/white-papers/operating-model-agentic-teams/.

12 Justin H. Kuiper, CISSP, "Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework," Non Sequitur Publishing, 2026, v1.0-preprint,

Skipjack (Paper 5) is the Protocol layer through which governance is operationally exercised.[13] This paper addresses what the Protocol requires of the Application domain of the agentic substrate.

Framework — HGC³AE² (Paper 1)
the organizing doctrine — what governance requires
governance flows down ▼
Protocol — Skipjack (Paper 5)
authority structure, role taxonomy, handoff discipline — how governance is operationally exercised
governance flows down ▼
Harness — Harnessed agents
LangGraph · AutoGen · CrewAI · MCP · OpenAI Agents SDK + LLM + tools + memory
governance flows down ▼
Application — The user-facing work the system performs — where governed output lands

What flows down to the substrate is concrete. From the Framework layer (Paper 1), the substrate inherits three obligations. The substrate must provide HG curation surfaces — memory architectures that record curation lineage, evidence-retention primitives that preserve what the human operator authorized. The substrate must carry the C³ context contract — manifest serialization, transport, and deserialization at every handoff. The substrate must support AE² admission, execution, and evidence as three distinct phases with substrate-side hooks at each transition. From the Protocol layer (Paper 5), the substrate inherits four mechanisms. The role taxonomy (Governance Layer / Orchestrator / Agentic Execution Layer) must be expressible as substrate-side authority partitioning, not just a labeling convention. The HITL control points must be invocable as interrupts that pause execution and route to the operator. The context integrity manifest must persist and propagate. The feedback loops must close through substrate-side hooks that surface SOP changes to every active agent context. From the Operating Model (Paper 6 §6.1),[14] the substrate inherits five concrete contracts: standup-equivalent telemetry surfacing, task admission gating, multi-context partition isolation, drift detection telemetry, and SOP propagation hooks. Each is a substrate-side primitive in this paper's specification.

What the substrate surfaces upward, in turn, is the evidence chain that makes governance auditable. The audit-grade evidence chain — every admission, every execution, every output reconstructable from substrate-stored artifacts — is the upward-flowing surface that the Protocol and Framework layers require. The capability inventory — the per-agent, per-envelope record of what the agent was authorized to do — surfaces upward to enable the orchestrator's admission decisions and the operator's completion reviews. The partition manifest — the record of which

---

https://nonsequitur.tech/white-papers/hgc3ae2/.

13 Justin H. Kuiper, CISSP, "Agile Scrum, Agentics, and the Skipjack Protocol: Governed Collaboration Between Human Judgment and Agentic Execution," Non Sequitur Publishing, 2026, v1.0-preprint, https://nonsequitur.tech/white-papers/skipjack-protocol/.

14 Kuiper, "Operating Model for Agentic Teams," §6.1 (Implications for Harness-Based Implementations).

cognitive modes are isolated and what re-unification points exist — surfaces upward to support the operating model's multi-account discipline. The blast-radius bound — the maximum harm an admission failure can produce given the substrate's isolation guarantees — surfaces upward as the precondition the Governance Layer needs in order to grant authority responsibly.

The harness landscape is the substrate the paper's specification operates over. LangGraph,[15] AutoGen,[16] CrewAI,[17] the Model Context Protocol,[18] the OpenAI Agents SDK,[19] and the peer-reviewed multi-agent frameworks (AutoGen, MetaGPT,[20] ChatDev[21]) supply the implementation primitives a substrate may compose. The paper does not endorse a specific composition. The substrate-research persona maintains a working dossier of harness-by-contract fitness in yks-build #73 / yks-spine-binder #227; that dossier feeds substrate-selection decisions for individual deployments. What this paper specifies is the contracts the chosen substrate must honor, not the harness that satisfies them. The contracts are substrate-independent; the satisfaction is implementation work.

---

2. AGENT ANATOMY: MEMORY, REASONING, AND TOOL ACCESS

2.1 WHY AGENT ANATOMY IS A GOVERNANCE SURFACE

Every agent provides three primitives: memory (a place to store and retrieve information across reasoning steps and across executions), reasoning (a loop that selects actions given current context and goals), and tool access (the means by which reasoning produces effects on the world). The peer-reviewed literature treats each as a capability surface — memory as the

---

15 LangChain, "LangGraph Documentation," 2024–2026, https://langchain-ai.github.io/langgraph/. (No peer-reviewed publication as of this writing.)

16 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155 (2024), https://arxiv.org/abs/2308.08155.

17 CrewAI, "CrewAI Documentation," 2024–2026, https://docs.crewai.com/. (No peer-reviewed publication as of this writing.)

18 Anthropic, "Model Context Protocol Specification," 2024, https://modelcontextprotocol.io/.

19 OpenAI, "OpenAI Agents SDK Documentation," 2024–2026, https://platform.openai.com/docs/. (No peer-reviewed publication as of this writing.)

20 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, https://arxiv.org/abs/2308.00352.

21 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, https://arxiv.org/abs/2307.07924.

substrate for long-horizon reasoning,[22][23] reasoning as the locus of intelligence,[24][25] tools as the means by which language-model competence reaches non-language tasks.[26] The capability framing is correct as far as it goes.

The capability framing is incomplete for governance purposes. Each of the three primitives is also a governance surface. Memory must record curation lineage — which facts the agent holds because the operator ratified them, which because the agent inferred them, which because a tool returned them. Reasoning must consult the admission gate — actions proposed by the reasoning loop must pass through substrate-side admission before they execute, regardless of the reasoning loop's confidence in their correctness. Tool access must be capability-bounded — the tools the agent can invoke at any moment are determined by the envelope it is executing under, not by the harness's global tool registry.

A substrate that exposes these primitives as capability features only — the agent has a memory, the agent reasons, the agent uses tools — is capability-rich and governance-thin. A substrate that exposes them as governance surfaces — the agent's memory carries lineage, its reasoning consults the gate, its tool calls are scoped — is governance-fit. The distinction is structural, and it determines whether the protocol layer above can be enforced or only suggested.

2.2 MEMORY ARCHITECTURES AND THEIR GOVERNANCE OBLIGATIONS

The agent memory landscape has four distinct architectural roles, each with a distinct governance obligation that the substrate must support.

Working memory is the per-task context window. It holds the current task's input, the agent's interim reasoning state, the tools it has called so far, and the manifest under which the task was admitted. Working memory is volatile by design: when the task closes, working memory dissolves. Its governance obligation is twofold. First, it must carry the C³ manifest — the manifest is not separate metadata; it is part of working memory the agent reasons over. Second, it must not leak across admission boundaries: the next task's working memory begins

---

22 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv preprint arXiv:2310.08560 (2023), https://arxiv.org/abs/2310.08560.

23 Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein, "Generative Agents: Interactive Simulacra of Human Behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, 2023, https://arxiv.org/abs/2304.03442.

24 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023, https://arxiv.org/abs/2210.03629.

25 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2303.11366.

26 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2302.04761.

fresh, with a new manifest, on a new admission decision. A substrate that retains working memory across tasks for performance reasons (a “session” abstraction that survives multiple admissions) introduces a contamination risk that defeats admission isolation.

Episodic memory is the across-task recall within a persona. When the substrate-research persona returns to a project after several days, episodic memory is what carries forward “we tried this last week and it failed for these reasons.” The governance obligation is curation lineage: every episodic-memory entry must carry a record of how it entered the persona’s belief — was it ratified by the operator, recorded as an agent inference, or extracted from a tool return — and a record of who authorized its retention. A substrate that records episodic events without lineage is recording experience but not curated belief; the persona’s later reasoning will treat ratified positions and unratified inferences as equally authoritative, which is a governance failure with downstream consequences.

Semantic memory is the persona-level long-lived knowledge: the working SOPs, the canonical positions, the persistent rules under which the persona operates. Semantic memory is the persona’s ratified worldview. The governance obligation is SOP-versioning with acknowledgement gating: every entry in semantic memory carries the version of the SOP under which it was admitted, and updates to an SOP propagate as version-bumps that require explicit acknowledgement before the persona resumes admission. A substrate that allows semantic memory to update silently — an SOP change reaches the persona’s belief store without an acknowledgement event — fails the operating-model contract that admission gates personas owing SOP acknowledgement (Paper 6 §6.1).

Persistent memory across cycles is the cross-cycle accumulation: the multi-month or multi-quarter historical record. The portfolio’s editorial-governance persona carries persistent memory of every paper’s draft history, every retrospective’s findings, every drift-report reconciliation. The governance obligation is HG-authorized archive, summarize, and forget operations. Persistent memory accumulates indefinitely if not curated, and unbounded accumulation is itself an epistemic-constraint failure mode (the persona’s belief store saturates beyond useful discrimination, per Paper 2). The substrate must surface curation operations to the operator: which entries persist, which are summarized, which are archived, which are forgotten. The substrate cannot make these decisions; it must support the operator’s making them.

The four architectural roles are not exclusive. Some substrates collapse working and episodic memory into a single per-session context. Others factor episodic and persistent memory into a unified “long-term memory” abstraction. The factoring is implementation detail. What matters for the governance contract is that each obligation — manifest carriage, curation lineage, SOP acknowledgement gating, HG-authorized lifecycle — is honored regardless of how the memory layers are factored. MemGPT[27] and the Generative Agents architecture[28] are the most-cited peer-reviewed substrates for the layered model; both can be made governance-fit but neither does so out of the box.

---

27 Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” arXiv preprint arXiv:2310.08560 (2023), https://arxiv.org/abs/2310.08560.

28 Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein, “Generative Agents: Interactive Simulacra of Human Behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST ’23)*, 2023, https://arxiv.org/abs/2304.03442.

2.3 REASONING LOOPS AND SCOPE BOUNDING

The agent's reasoning loop is the locus of action selection. Several reasoning patterns appear repeatedly in the peer-reviewed substrate literature, and each carries a distinct scope-bounding governance obligation.

ReAct loops[29] interleave reasoning and action: the agent reasons about what to do, takes an action, observes the result, reasons again. The pattern is structurally simple and operationally common. Its governance obligation is that each *act* step consult the admission gate. The substrate must intercept the action before it reaches the tool or downstream agent, check it against the admission envelope, and either admit or refuse. A ReAct loop that calls tools directly without substrate-side admission interception is performing scope-deviating actions whose cumulative effect exceeds the admission envelope; the reasoning step makes the action plausible, but plausibility is not authorization.

Plan-then-execute patterns separate the planning phase from the execution phase: the agent produces a plan (a sequence of intended actions) and then executes it. The pattern is common in goal-directed agents and in agentic workflow systems. Its governance obligation is that the plan itself is an admission artifact. The orchestrator admits the plan, not the individual steps, but admission of the plan binds execution to the planned shape. Mid-execution deviations from the plan re-trigger admission. A plan-then-execute loop that allows the executor to deviate from the plan without re-admission has converted the admission decision into a recommendation.

Tool-augmented reasoning (the Toolformer pattern[30]) places tool calls inside the reasoning generation: the language model generates text that includes tool invocations, the substrate executes the tools, the substrate splices the results into the generated text. The pattern is elegant and produces high-quality reasoning chains. Its governance obligation is that tool calls inside generation are admission events, not reasoning side-effects. A tool-augmented reasoning system that treats tools as part of the language-model API rather than as distinct admission events fails the capability-scoping contract: any tool the harness has registered becomes invokable by any reasoning chain, regardless of envelope.

Reflexion[31] adds self-correction across reasoning episodes: when an agent's first attempt fails, the agent reflects on the failure and tries again. The pattern is powerful for tasks where the success criterion is verifiable. Its governance obligation is that reflection must not extend scope. A reflective agent that identifies a scope-deviation as a failure and "corrects" by adopting the deviated scope as the new target is converting scope deviation into scope expansion. The

---

29 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023, https://arxiv.org/abs/2210.03629.

30 Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2302.04761.

31 Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023, https://arxiv.org/abs/2303.11366.

substrate must distinguish between corrections within the envelope (acceptable) and corrections that redefine the envelope (require operator escalation).

The four patterns are not mutually exclusive. A working agent often combines ReAct-style action selection inside a plan-then-execute outer loop, with tool-augmented reasoning at the leaves and Reflexion-style self-correction at the iteration boundary. The combination is fine; the governance obligation is that admission interception is present at every level — every action, every tool call, every reflection — and that scope is bounded at the envelope, not at the reasoning loop's confidence.

2.4 TOOL ACCESS AS CAPABILITY ADMISSION

Tool access is the means by which reasoning produces non-language effects. Three patterns recur in the substrate landscape.

Direct tool call. The agent invokes a tool API directly through harness-provided wrappers. The pattern is simple and fast. Its governance obligation is that the invocation pass through a substrate-side admission hook before the tool executes. The hook checks the envelope, the manifest, and the SOP acknowledgement state before allowing the tool call to reach the tool. A direct tool call without admission interception is a tool invocation outside the governance flow.

MCP-mediated tool call. The Model Context Protocol[32] specifies a standardized server interface for tool exposure: an MCP server exposes tools through a defined protocol, and harnesses connect to MCP servers to access the tools. The pattern is increasingly common because it decouples tool development from harness development. Its governance obligation is that MCP servers integrate with the substrate's admission and evidence channels. An MCP server that bypasses the substrate's admission flow — accepting and executing requests without going through the substrate's gate — is a substrate-level failure regardless of what the harness around it does.

Capability-gated tool call. The substrate issues a capability token at admission time; the tool invocation requires the token; the tool refuses calls without a valid token. The pattern is the most governance-aligned of the three. The token's scope is bounded by the envelope, the token's lifetime is bounded by the execution duration, and the token cannot be acquired outside the admission flow. Capability tokens are an old idea in operating-system security[33][34] and applying them to agentic tool access is a governance-aligned modernization of a well-understood primitive.

Most production substrates today use a mix of direct and MCP-mediated calls. Capability-gated calls are the rarer pattern but the most defensible governance-wise. The substrate-selection question is not which pattern is dominant in a candidate harness, but how much governance the candidate's tool-access pattern enables. A harness whose tool calls all flow through MCP servers that integrate with a substrate-level admission hook can be governance-

32 Anthropic, "Model Context Protocol Specification," 2024, https://modelcontextprotocol.io/.

33 Jerome H. Saltzer and Michael D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE* 63, no. 9 (1975): 1278–1308, https://doi.org/10.1109/PROC.1975.9939.

34 Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano, "Passwords and the Evolution of Imperfect Authentication," *Communications of the ACM* 58, no. 7 (2015): 78–87, https://doi.org/10.1145/2699390.

fit. A harness whose tool calls are direct API invocations from inside the language model's generation, without substrate-side interception, cannot.

2.5 THE AGENT ANATOMY CONTRACT

An agent is governance-fit when three conditions hold. Its memory carries lineage: every fact the agent acts on can be traced to the curation event that admitted it. Its reasoning consults the gate: every action the reasoning loop proposes passes through substrate-side admission before execution. Its tool calls are scoped: the capabilities invokable in any execution are bounded by the envelope under which the execution was admitted.

The conditions are simple and they are jointly necessary. Memory without lineage produces an agent that acts confidently on unverified beliefs. Reasoning without admission consultation produces an agent that performs scope-deviating actions whenever the reasoning chain renders them plausible. Tool access without scoping produces an agent whose capabilities are determined by what the harness exposes rather than by what the operator authorized.

The substrate's job is to make these conditions invocable as primitives, not as conventions. A substrate that documents memory-lineage best practices but does not enforce them is conventional. A substrate that requires every memory write to record lineage, every reasoning action to traverse the gate, every tool call to present a token — and refuses operations that lack the metadata — is structural. The protocol layer can build on the structural substrate; it cannot reliably build on the conventional one.

2.6 WORKED EXAMPLE: THE FOUR SHARED MODEL POOLS

The agentic-substrate posture stated in Paper 5 §10 and operationalized in the working portfolio is open-source-only, shared-pool, locally-hosted. The posture's substrate-side expression is the four-pool architecture: distinct inference pools serve distinct agent classes. Command pool serves orchestration agents whose reasoning is short, structured, and routing-focused. Code pool serves substrate-research and engineering agents whose reasoning is iterative and produces structured artifacts. Narrative pool serves editorial-governance and content-pipeline agents whose reasoning is voice-sensitive and lineage-heavy. Analysis pool serves cross-domain reasoning that operates over the other pools' outputs.

The four-pool architecture is a governance design, not a capacity-management one. Each pool has a distinct set of canonical positions (the SOPs the agents in the pool honor), a distinct memory store (the persistent and episodic memory the pool's agents draw from), and a distinct routing policy (rule-based for command, capability-scored for code, persona-bound for narrative, latency-bounded for analysis). The pool boundaries are partition-isolation primitives at the substrate level: a code-pool agent cannot read the narrative pool's curation history because the substrate does not expose it. The isolation is structural, and it directly satisfies the multi-context partition isolation contract of Paper 6 §6.1.

The shared-pool aspect is operational economy: each pool serves multiple agents from the same persona class, sharing inference infrastructure cost. The locally-hosted aspect is governance economy: the inference happens on infrastructure the operator controls, which preserves manifest propagation and credential scope across the inference call. Proprietary API inference defeats both: the manifest does not travel with the API call (the API does not know what to do with it), and credential scope is bounded by the API key rather than by the envelope.

The yks-llm-gateway[35] live deployment on K3s implements this posture; the substrate-research persona operates the gateway and the four pools.

The four-pool example is illustrative, not prescriptive. A different deployment might require five pools, or three, or a different categorization. What is prescriptive is the substrate property: pool boundaries are partition primitives, pools are routed by policy, and the policies are themselves governance artifacts. A single shared inference endpoint serving all agent classes through prompt-side persona switching does not satisfy the substrate contract regardless of how careful the prompts are.

---

3. ORCHESTRATION PATTERNS: SINGLE-AGENT, MULTI-AGENT, AND SWARM

3.1 ORCHESTRATION TOPOLOGY DETERMINES AUTHORITY DISTRIBUTION

Orchestration topology is conventionally treated as a capability choice. A team chooses single-agent or multi-agent or swarm based on which topology best matches the task throughput, the parallelism the task admits, the coordination overhead the team can afford. The choice is presented as a tuning decision: which configuration produces the best results for this task class.

The capability framing misses the structural property. Topology is also — and for governance purposes, primarily — an authority choice. Each topology distributes Governance Layer / Orchestrator / Executor authority differently, and the distribution determines what governance is implementable.[36] The Skipjack Protocol's three-tier role structure assumes that authority can be partitioned across nodes; some topologies admit that partition cleanly, others collapse it, others fragment it. A team that selects topology by capability and discovers the authority implications afterward has chosen its governance structure without realizing it.

Three topologies are common in the production substrate landscape, and each has a characteristic authority distribution that the substrate selection must honor. Single-agent collapses the orchestrator into the executor. Multi-agent with explicit role typing separates them but requires substrate-side enforcement of the typing. Swarm distributes authority across peers and requires consensus mechanisms the protocol layer must specify. The choice among them is governance-consequential, and the consequences are not optional.

3.2 SINGLE-AGENT ORCHESTRATION

Single-agent orchestration is the simplest case. One agent receives the task, executes it, returns the result. There is no orchestrator distinct from the executor; the agent is both. The pattern is common in early agentic adoption because it is cognitively simple and operationally fast — and because the harness landscape supports it well. ReAct-style loops, plan-then-execute agents, and many tool-using assistants are single-agent in topology.

---

35 The yks-llm-gateway is the working portfolio's local-inference gateway, deployed on K3s, exposing the four shared model pools (command, code, narrative, analysis) referenced in this paper and in Paper 5 §10. Operations are documented in the substrate-research persona's working dossier (yks-build #73 / yks-spine-binder #227).

36 Kuiper, "Skipjack Protocol," §2 (The Role Remap from Scrum to Skipjack).

The authority implication is that the orchestrator has been collapsed into the executor. The Governance Layer must specify scope at admission — the envelope must be complete, the manifest must be attached, the SOP acknowledgement state must be verified — because once the agent begins executing, the only enforcement boundary is the agent's own reasoning loop. The substrate has no separate node to interpose between the agent's reasoning and its actions; the same node that selects the action also performs it.

Scope drift within a single execution is the characteristic failure mode. A capable agent reasoning across a long task will, over enough steps, propose actions that cumulatively exceed the envelope while no single action obviously violates it. Each tool call seems plausible given the prior state; each reasoning step seems consistent with the goal as the agent has interpreted it; and the executed work, audited after the fact, contains scope-deviating outputs that no single decision authorized. The drift is not malice; it is reasoning entropy.

Three substrate-side mitigations apply. First, *bounded reasoning loops*: the substrate caps the loop depth. After N steps, the agent must return for re-admission rather than continue. The depth bound is a governance parameter, not a tuning parameter. Second, *in-loop admission re-checks*: each action the reasoning loop proposes is re-checked against the envelope before execution. The cost is interception overhead per step; the benefit is that scope drift cannot accumulate silently because the gate fires on every action. Third, *Reflexion-style scope re-verification at iteration boundaries*: when the agent self-corrects, the substrate verifies that the correction remains within the envelope. The substrate refuses corrections that re-define the envelope; those become operator escalations.

When to use single-agent topology: tasks whose scope is small enough to be specified atomically, whose reasoning chain is short enough that the bounded-loop and re-check overhead does not dominate, and whose risk profile is low enough that the collapsed orchestrator authority is acceptable. The class of tasks that meet all three conditions is real and operationally large; for many production tasks, single-agent is the right answer. The class is not, however, universal, and substrate selection that defaults to single-agent for all tasks is selecting away from the topologies governance most clearly supports.

3.3 MULTI-AGENT ORCHESTRATION WITH EXPLICIT ROLE TYPING

Multi-agent orchestration distributes the work across two or more agents with distinct, named roles. The peer-reviewed instances of the pattern that have been most influential are AutoGen,[37] MetaGPT,[38] and ChatDev.[39] AutoGen specifies a multi-agent conversational

37 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint arXiv:2308.08155 (2024), https://arxiv.org/abs/2308.08155.

38 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, https://arxiv.org/abs/2308.00352.

39 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, https://arxiv.org/abs/2307.07924.

framework in which agents with assigned roles negotiate tasks through structured exchange. MetaGPT adopts standardized operating procedures from human software-engineering practice and assigns agents to roles (product manager, architect, engineer) with role-specific responsibilities. ChatDev runs a waterfall-style decomposition across CEO / CTO / programmer / reviewer agents.

The authority implication is that the orchestrator becomes a separate node with admission authority over the executors. Tasks pass from the orchestrator to the executor through an admission boundary; the executor's reasoning loop does not negotiate scope directly with other executors. The role typing is the structural mechanism by which authority is distributed: a "reviewer" agent is not a "programmer" agent, not because the underlying model is different, but because the substrate restricts what each role may do. A reviewer's actions are gated by reviewer-role envelopes; a programmer's actions by programmer-role envelopes.

The pattern's characteristic failure mode is collusion. Two executors with the same underlying model, sharing context, and exchanging messages can — and over enough executions, will — agree informally to expand scope. The agreement is not adversarial; it is the same reasoning entropy that produces single-agent scope drift, now distributed across two reasoning chains that reinforce each other. A programmer agent suggests an out-of-scope refactor; the reviewer agent, evaluating the change against the in-scope criteria, finds it acceptable; the cumulative effect is scope expansion neither agent individually proposed.

Substrate mitigation requires that role typing be enforced substrate-side, not conventionally. AutoGen's role-typing is largely conventional: roles are prompt-defined, and the substrate does not enforce role-bounded capabilities. MetaGPT enforces some role properties through its SOP-driven structure but does not gate inter-agent scope changes through a substrate-side authority. ChatDev's waterfall structure enforces sequential role hand-offs but does not prevent role collusion within a phase. None of these is a substrate failure that the framework's authors did not address; it is a substrate property that governance-grade deployment requires and that the peer-reviewed substrates have not yet been built for.

The substrate-side enforcement requires three primitives. First, *role-bounded capability tokens*: each role's tokens authorize a different capability set, and an agent acting in one role cannot acquire another role's tokens. Second, *orchestrator-mediated inter-agent messages*: messages from one agent to another pass through the orchestrator's audit channel before delivery. The orchestrator is not necessarily a heavyweight reasoning node; it can be a thin gate that records and admits messages. Third, *envelope-bounded role assignment*: an agent's role for a given execution is bound by the envelope under which it was admitted, not by a session-level role assignment that persists across admissions. A reviewer agent invited into a new envelope as a programmer is admitted with programmer capabilities, and its prior reviewer role does not leak through.

When to use multi-agent orchestration: tasks that decompose naturally into role-distinct phases, throughput requirements that benefit from parallelism within an envelope, and risk profiles for which role-bounded authority is the appropriate granularity of governance. The peer-reviewed substrates above are useful starting points; the substrate-side governance work to make them governance-fit is engineering, not research.

3.4 SWARM ORCHESTRATION

Swarm orchestration distributes work across many peer agents with no single orchestrator. Decisions are made by consensus protocols: voting, leader election, gossip-based agreement, quorum requirements. The pattern is rare in production agentic systems but appears in research prototypes and in some specialized domains (distributed sensing, decentralized coordination).

The authority implication is severe. The Governance Layer cannot route through a single orchestrator because there is no single orchestrator. Authority must be distributed across the swarm, which requires that admission decisions, manifest verifications, and HITL escalations be made through consensus mechanisms rather than through a single authoritative node. The Skipjack Protocol's role taxonomy was not designed for distributed authority; making it work in a swarm requires either appointing one swarm member as the de facto orchestrator (which collapses the swarm pattern toward multi-agent) or specifying consensus rules for governance decisions (which adds substantial complexity to the protocol layer).

The characteristic failure mode is byzantine behavior. One swarm member proposes a scope expansion; under sufficiently lax consensus rules, others ratify; the expansion proceeds without operator authorization. The failure is not necessarily due to malice — a swarm member that has reasoned itself into believing the expansion is legitimate is byzantine in effect even if it is not byzantine in intent. The classical distributed-systems literature on byzantine fault tolerance[40] applies, with its trade-offs between consensus latency and fault tolerance.

Mitigation requires substrate primitives that the agentic-substrate landscape does not yet provide as standard. Cryptographic admission tokens — tokens that bind admission to a Governance-Layer signature, such that a swarm cannot self-admit work — are one. Quorum requirements for scope changes — a scope expansion requires not just consensus but Governance-Layer-signed authorization beyond a threshold — are another. These are well-understood techniques in distributed-systems security but are not part of the standard agentic-substrate toolkit.

When to use swarm topology: very rarely in governance-grade contexts. The consensus overhead typically outweighs the throughput gain, the byzantine-tolerance machinery is heavy, and the authority-distribution challenges are real. The pattern's appropriate domains are those where centralization is not just inefficient but structurally unavailable — geographically distributed sensing, edge environments without persistent connectivity, multi-organization collaborations where no single orchestrator can be appointed. For most enterprise agentic deployments, swarm is the wrong answer; multi-agent with explicit role typing is the right one.

3.5 CENTRALIZED VS. PEER-TO-PEER TOPOLOGY

A topology choice that cuts across the single-agent / multi-agent / swarm taxonomy is whether the topology is centralized or peer-to-peer. Centralized: there is a distinguished orchestrator node, and all admission decisions flow through it. Peer-to-peer: any node can admit (subject to consensus rules), and admission decisions are negotiated.

For governance-grade deployments, centralized is the default and the recommendation. The reasons are structural. Partition isolation is structurally easy in a centralized topology because the orchestrator is the boundary between partitions. Manifest propagation is structurally easy because the orchestrator is the manifest's authoritative source for each handoff. HITL escalation is structurally easy because the operator's escalation point is the orchestrator. A centralized topology aligns the substrate's authority structure with the protocol layer's authority structure; it is not a coincidence that Skipjack's role taxonomy maps cleanly onto centralized topologies and uneasily onto peer-to-peer ones.

Peer-to-peer is justified when the centralized orchestrator becomes a bottleneck or a single point of failure that the deployment cannot tolerate. The bottleneck case is rare in practice; orchestrator nodes scale horizontally, and most admission throughput needs are met by adding

40 Leslie Lamport, "The Part-Time Parliament," *ACM Transactions on Computer Systems* 16, no. 2 (1998): 133–169, https://doi.org/10.1145/279227.279229.

orchestrator capacity rather than by removing the orchestrator. The single-point-of-failure case is more common in resilience-critical deployments — military, life-safety, edge environments where a connectivity loss to the orchestrator must not stop the work. These are the genuine peer-to-peer use cases, and they are sufficiently specialized that the deployment can afford the substrate-engineering work to make peer-to-peer governance-fit.

3.6 STATEFUL VS. STATELESS EXECUTION

A second cross-cutting choice is whether execution is stateful or stateless. Stateful: an agent retains context across tasks, often through a session abstraction the harness provides. Stateless: an agent receives full context per task; nothing persists across admissions.

The capability advantage of stateful execution is throughput on related work. An agent that retains the manifest, the SOPs, and the working memory from the prior task can begin the next task without paying the context-construction cost. For tasks that arrive as related sequences (debugging a single feature across multiple admissions, editing a single document across multiple revisions), the throughput gain is meaningful.

The governance risk of stateful execution is context contamination across admission boundaries. The state that persists from task A may contain assumptions, beliefs, or partial reasoning that task B is not authorized to inherit. The C³ context contract[41] is per-task; a stateful agent is, in effect, executing task B under a context that includes task A's state. If task A and task B were admitted under different envelopes, the contamination violates the envelope boundary.

The Skipjack Protocol's manifest-on-every-task discipline (§4.2) leans toward stateless execution: every task gets a fresh manifest, a fresh admission decision, a fresh context. The stateless lean has a cost — repeated context construction — that the substrate can mitigate without compromising admission discipline. *Substrate-side caching* is the right pattern: the substrate caches computation that is verifiably independent of admission state (model loading, tool registration, infrastructure connectivity) and rebuilds the parts of context that are admission-dependent (manifest, envelope, SOP-acknowledgement state) on every task. The cache is operational; the manifest is governance. Mixing the two — caching the manifest for performance — is the failure mode.

3.7 OUTPUT NORMALIZATION AND NEXT-HOP ROUTING

Two final substrate-layer concerns close the orchestration pattern treatment. First, *output normalization*: the orchestrator receives outputs from executing agents in whatever format the agent produces, and converts them into a substrate-canonical format before evidence capture. The normalization is governance-consequential: an output that the orchestrator cannot parse is an output the orchestrator cannot audit, which means the AE² evidence chain is broken. The substrate must specify the canonical format and refuse outputs that do not conform — or, more precisely, refuse to record them as evidence-bearing without normalization.

Second, *next-hop routing*: based on the normalized output, the orchestrator decides whether to admit a follow-up task, escalate to the operator, or close the chain. The next-hop policy is a substrate-layer concern, not an inter-agent concern. Executing agents do not negotiate next steps among themselves; they return outputs to the orchestrator, and the orchestrator routes. A substrate that allows agents to direct work to each other without orchestrator-mediated routing has converted the orchestration pattern from multi-agent to swarm without reaping the swarm's coordination benefits or paying the swarm's consensus

41 Kuiper, "Skipjack Protocol," §4 (Context Integrity and Epistemic Continuity).

costs. The pattern is observable in some production deployments and is generally a substrate failure mode, even when the routing decisions themselves are reasonable.

The orchestration pattern selection is therefore not a tuning decision. It is a substrate-design decision whose downstream consequences run all the way through the protocol layer to the operating model. A substrate selected for capability that does not support the orchestration pattern the protocol requires will produce an operating model whose governance is implementable only as convention. Substrate selection should begin with the pattern the protocol requires and proceed to the harness candidates that support it; not the other way around.

---

4. GOVERNANCE INTEGRATION: EMBEDDING HITL AND SKIPJACK AT THE SUBSTRATE LEVEL

4.1 WHY GOVERNANCE INTEGRATION MUST BE SUBSTRATE-LAYER

Governance specified above the substrate — in the protocol layer's prose, in the orchestrator's application code, in the operator's working rules — is enforced only as long as the agent honors it. A capable agent that can route around an above-substrate gate will, over enough executions, do so. The route-around is rarely intentional. It is the consequence of reasoning loops that, given a goal and a tool registry, will find paths the human reviewer would not have authorized but the substrate did not block. The literature on AI safety has discussed this property under several names: specification gaming, instrumental convergence, reward hacking.[42] The names differ; the underlying phenomenon is the same. Capability finds paths.

The structural mitigation is to make the gate the only path. Substrate-layer enforcement places the governance check at a point the agent cannot bypass: the substrate is the only path from agent reasoning to agent action, and the substrate is the locus of the check. The agent does not honor the gate; the agent encounters the gate. The distinction between honoring and encountering is the distinction between conventional governance and structural governance.

The argument generalizes beyond admission. Every governance contract in the operating model — admission gating, evidence capture, manifest propagation, HITL escalation, SOP propagation — has the same property: it is enforceable only if the substrate makes the enforcement structural. A protocol-layer specification of HITL control points without substrate-layer enforcement is a specification of where reviews would happen if the agents requested them, which they will not reliably do. The peer-reviewed AI-safety literature has been clear on this point for over a decade,[43] and the substrate-design implications are direct: governance must be at the substrate, or it will not, in the long run, be governance.

---

42 Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané, "Concrete Problems in AI Safety," arXiv preprint arXiv:1606.06565 (2016), https://arxiv.org/abs/1606.06565.

43 Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané, "Concrete Problems in AI Safety," arXiv preprint arXiv:1606.06565 (2016), https://arxiv.org/abs/1606.06565.

4.2 THE ADMISSION HOOK

The admission hook is the substrate's pre-execution callback that fires before the agent receives a task. It is the substrate-side instantiation of the Skipjack §3.4 control point[44] and the Paper 6 §6.1 task-admission-gating contract.[45]

The hook's inputs are five: the task spec (what the agent has been asked to do), the parent envelope (the Governance-Layer-approved scope under which this task was decomposed), the agent identity (which persona, which capability set, which role-binding), the manifest version (which Skipjack context integrity manifest applies), and the SOP acknowledgement state (whether this persona has acknowledged the current SOP versions for this task class).

The hook's outputs are three: admit, refuse, escalate. Admission proceeds to execution. Refusal returns the task to the orchestrator with a refusal reason, which the orchestrator may record as evidence and either re-decompose, re-admit under different conditions, or surface to the operator. Escalation pauses the admission and surfaces the decision to the operator for HITL adjudication, with the task held in a non-executing state until the operator returns.

The substrate requirement is that the hook is the only path. If an agent can execute a task without traversing the hook — through a back-channel API, through a session-cached context, through a tool-call that happens before the substrate's gate fires — admission discipline is not enforced. The hook must be at the substrate boundary, and the substrate must not provide alternative routes.

Most production harnesses today provide *something like* an admission hook but not exactly an admission hook. CrewAI's process manager intercepts task delegation but does not enforce envelope-bounded admission; LangGraph's node functions are intercept points but are agent-defined rather than substrate-imposed; the OpenAI Agents SDK's tool-call interception fires after the agent has decided to call the tool, which is too late for envelope-level admission decisions. None of these is a fundamental defect; each is an absence of the specific governance primitive that admission gating requires. Composing the missing primitive on top of these harnesses is engineering work that the substrate-research persona's dossier should track per harness.

4.3 THE EVIDENCE CHANNEL

The evidence channel is the substrate's first-class output channel for AE² evidence. It is distinct from agent stdout, distinct from tool return values, distinct from the harness's diagnostic logging. It is structured: every execution emits a record with seven fields — input received, action executed, output produced, deviations from envelope, manifest version honored, handoff state, evidence checksum.

The channel's purpose is auditability. The orchestrator audits the channel before declaring a task complete, and the cycle's accumulated evidence is what the operator's completion review[46] consults. A substrate that conflates evidence with diagnostic logging makes the audit a forensic exercise: the operator must reconstruct, from heterogeneous logs, what actually

---

44 Kuiper, "Skipjack Protocol," §3.4 (Mechanism 3 — Human-in-the-Loop Control Points) and §3.3 (Mechanism 2 — Structured Task Decomposition and Routing).

45 Kuiper, "Operating Model for Agentic Teams," §6.1 (Implications for Harness-Based Implementations).

46 Kuiper, "Skipjack Protocol," §5.1 (Why HITL Must Be Structural) and §5.2 (Control Point Taxonomy).

happened. A substrate that emits structured evidence on a dedicated channel makes the audit structural: the operator (or an auditing persona) reads the evidence directly.

Three substrate properties make the evidence channel governance-fit. First, *write-only from the agent's perspective*: the agent emits evidence but cannot read or modify previously emitted evidence. The substrate prevents an agent from constructing evidence that contradicts what the agent actually did. Second, *append-only at the channel level*: evidence is not edited or deleted after emission. Corrections take the form of additional evidence records that reference the prior record. Third, *per-task isolation*: evidence from task A is not visible to task B except through the orchestrator's audit. The orchestrator may surface prior evidence to a follow-up task as part of context construction, but the surfacing is an admission-time decision, not an automatic data-flow.

The channel's storage is substrate-side, not agent-side. An agent that "logs evidence" by writing to its own memory is providing self-reported evidence, which the orchestrator cannot verify against the actual execution. The substrate captures evidence at the moment of execution — the input the agent received from the substrate, the action the agent emitted to the substrate, the output the substrate produced from the action — because the substrate is the witness. Self-reported evidence is the failure mode that turns the AE² return into a narrative the agent constructs about its own behavior; substrate-witnessed evidence is the discipline that makes the AE² return audit-grade.

4.4 MANIFEST PROPAGATION

The Skipjack context integrity manifest[47] is the artifact that travels with each task across handoffs. The manifest carries the original Governance-Layer intent, the explicit exclusion records (what scope was deliberately cut from the envelope), the constraint registry (what rules apply to this work), and the handoff receipt log (which nodes have processed this manifest and what they did). It is the carrier of context integrity through the execution chain.

The substrate requirement is that manifest propagation be substrate-side. Serialization (the manifest is converted to a transport format), transport (the manifest is conveyed to the next node), and deserialization (the manifest is re-instantiated at the next node) are substrate operations, not agent operations. An agent does not synthesize, modify, or summarize the manifest. The substrate enforces this by treating the manifest as opaque from the agent's perspective: the agent reads the manifest as needed for its reasoning but cannot mutate it; the manifest the next node receives is the manifest the substrate transported, not whatever the agent produced.

The substrate-side requirement has three components. First, *deterministic serialization*: the same manifest produces the same serialized form, byte-for-byte, regardless of which node is serializing. Determinism is what makes the manifest's checksum a meaningful integrity check. Second, *integrity verification at deserialization*: the receiving node verifies the manifest's checksum before deserializing; a corrupted or modified manifest is refused. Third, *handoff receipt logging*: each node that processes a manifest emits a receipt record (which node, when, what action), and the receipts compose into the chain-of-custody trail that supports audit.

Most agentic substrates today do not transport manifests because the manifest concept is not native to the harness landscape. Skipjack §4.2 specifies the manifest as a governance artifact; the substrate-side instantiation is implementation work that the substrate-research persona's dossier identifies per harness. A harness that does not transport a manifest by default but exposes a serialization-capable wire format can be extended to do so; a harness that

47 Kuiper, "Skipjack Protocol," §4.2 (The Context Integrity Manifest in Practice).

flattens cross-node communication to bare strings cannot, without substrate engineering that effectively replaces the harness's transport layer.

4.5 HITL ESCALATION ROUTES

Skipjack §5 specifies three control point types[48]: approval gate, scope checkpoint, completion review. Each must be invocable from the agent's reasoning loop and from the orchestrator's admission decision. The substrate primitive that supports invocation is the *interrupt-style escalation hook*: a substrate-side mechanism that pauses execution, surfaces the decision to the operator, and resumes execution only on operator acknowledgement.

The interrupt is a substrate primitive because the alternative — having the agent or the orchestrator implement HITL through application-level polling, queueing, or scheduled review — fails the structural-enforcement test. Application-level HITL relies on the agent or the orchestrator to surface the decision; an agent that does not surface a borderline decision because it has reasoned itself into resolving the case directly is bypassing the control point. Substrate-level HITL fires at the substrate boundary, regardless of the agent's reasoning.

The substrate primitive has four properties. First, *non-bypassable*: an agent cannot continue execution past the escalation point until the operator has responded. The substrate enforces the wait. Second, *operator-routed*: the substrate has a routing policy for which operator (which Governance-Layer human, in multi-operator deployments) receives which escalation. The routing is part of the operator's authorization scope. Third, *time-bounded*: an escalation that the operator does not respond to within a configured window is automatically refused (with a specific refusal code), not silently admitted. Silent default-admit is a governance-failure mode that some harnesses today exhibit when their HITL implementation is convention rather than primitive. Fourth, *evidence-emitting*: the escalation event itself is an evidence record on the channel — when it fired, what it asked, who responded, what the response was.

In the working portfolio, the most-used HITL primitive is the approval gate fired at task admission. The operator approves envelopes; the substrate's admission hook fires for each task derived from the envelope; tasks within the envelope's scope admit without further escalation; tasks that surface scope-deviation triggers escalate to the operator for re-adjudication. The approval gate's load is bounded by the envelope authoring rate (Paper 6 §2.3) rather than by the task throughput rate, which is what makes the operating model scale.

48 Kuiper, "Skipjack Protocol," §5.1 (Why HITL Must Be Structural) and §5.2 (Control Point Taxonomy).

4.6 THE GOVERNANCE INTEGRATION MATRIX

The four primitives of §4.2 through §4.5 each enforce one or more of the four Skipjack mechanisms.[49] The mapping is direct.

Substrate primitive	Skipjack mechanism enforced	Paper-6 §6.1 contract supported
Admission hook (§4.2)	M2 (decomposition + admission), M3 (HITL control points)	Task admission gating
Evidence channel (§4.3)	M4 (feedback loops — closure depends on evidence audit)	Standup-equivalent telemetry surfacing
Manifest propagation (§4.4)	M1 (context integrity and epistemic continuity)	Multi-context partition isolation, Drift detection telemetry
HITL escalation (§4.5)	M3 (HITL control points)	Standup-equivalent telemetry, Task admission gating

The matrix is not orthogonal — several primitives support multiple Skipjack mechanisms, and several mechanisms require contributions from multiple primitives. The orthogonality is not the goal; the coverage is. A substrate that exposes all four primitives covers all four Skipjack mechanisms; a substrate missing any primitive leaves at least one mechanism un-enforced and at least one Paper-6 contract un-fulfilled.

49 Kuiper, “Skipjack Protocol,” §3.4 (Mechanism 3 — Human-in-the-Loop Control Points) and §3.3 (Mechanism 2 — Structured Task Decomposition and Routing).

4.7 WORKED EXAMPLE: THE OPEN-SOURCE / SHARED-POOL / LOCALLY-HOSTED POSTURE

The substrate posture stated in Paper 5 §10 (Domain 3) — open-source-only inference, shared-pool architecture, locally-hosted execution — is a substrate-design choice with direct governance integration consequences. The posture is not arbitrary; each element of it is a substrate-side commitment whose absence would defeat one or more of the §4.2–§4.5 primitives.

Open-source-only inference. The substrate-research persona operates the inference layer; the inference code is auditable, the model weights are inspectable, and the substrate can be modified to surface the governance hooks the protocol requires. Closed-source inference (proprietary API endpoints) defeats the integrity-verification step of manifest propagation: the manifest cannot be verified at the inference call because the inference endpoint does not expose the verification primitive. Auditability of the inference layer is a substrate property, and the open-source posture preserves it.

Shared-pool architecture. The four-pool design discussed in §2.6 — command, code, narrative, analysis — shares inference infrastructure across agents within a pool while preserving partition isolation across pools. The shared aspect is operational; the partition aspect is governance. A single shared inference endpoint serving all agent classes through prompt-side persona switching would defeat partition isolation; four pools with substrate-side routing preserve it.

Locally-hosted execution. The inference happens on infrastructure the operator controls. Manifest propagation is preserved across the inference call because the substrate controls both ends of the call. Credential scope is bounded by the envelope because the substrate issues credentials at admission and the inference layer honors them. Proprietary API inference defeats both: the manifest does not travel with the API call (the API does not know what to do with it), and credentials are the API key (which is per-deployment, not per-envelope, and is therefore unscoped from the substrate's perspective).

The yks-llm-gateway[50] live deployment is the working example. The gateway runs on K3s, exposes the four pools through governance-aware routing, and integrates with the substrate's admission hook, evidence channel, manifest propagation, and HITL escalation primitives. The gateway is small enough to be a working artifact rather than an aspiration; it fits on a single Kubernetes cluster and serves the portfolio's five personas. The implementation is not the contribution; the existence proof is. A substrate that exposes the four governance primitives is implementable, today, on commodity infrastructure.

4.8 WHAT GOVERNANCE INTEGRATION IS NOT

A note on what governance integration is not, because the substrate design space is muddled by terminology adjacent to governance that does not actually constitute it.

Governance integration is not *adding wrappers around agent capabilities*. A wrapper that intercepts a tool call to log it is observability; a wrapper that intercepts a tool call to refuse it under specific conditions is approaching admission, but only if the refusal is structural (the agent cannot route around it). Most "guard rail" libraries in the current substrate landscape are

50 The yks-llm-gateway is the working portfolio's local-inference gateway, deployed on K3s, exposing the four shared model pools (command, code, narrative, analysis) referenced in this paper and in Paper 5 §10. Operations are documented in the substrate-research persona's working dossier (yks-build #73 / yks-spine-binder #227).

observability with conditional refusal at the application layer; they are useful but not substrate-level governance.

Governance integration is not *responsible-AI prompting*. A system prompt that instructs the agent to follow specific rules — "do not access systems outside the user's scope," "do not produce content that violates the policy" — is conventional governance. The agent honors the prompt to the extent the prompt is salient in its reasoning; over enough executions, the salience erodes and the prompt is bypassed. Prompts are not enforcement; they are suggestion at the model's input layer.

Governance integration is *substrate-layer interception that the agent cannot circumvent*. The interception is structural — the substrate is the only path through which the agent can act, and the interception is at the substrate boundary. Whether the interception is implemented in Python, Rust, eBPF, or hardware is implementation detail. Whether the interception fires before, during, or after the action is design choice (different governance contracts require different timings). What is not negotiable is that the interception is unbypassable: the agent encounters the gate, rather than honors it.

---

5. RUNTIME ISOLATION AND SECURITY BOUNDARIES

5.1 ISOLATION AS A BLAST-RADIUS CONTRACT

Agent execution can produce harm. The harm may be intentional (an adversarial agent attempting to exfiltrate data), unintentional (a reasoning loop that proposes a destructive action because the action looks plausible given the goal), or compositional (several agents whose individually-bounded actions, taken together, exceed the operator's intended authorization). The substrate cannot eliminate the possibility of any of these. The substrate's job is to bound the consequences.

That bound is the *blast-radius contract*: the substrate establishes the maximum harm that an admission failure or an executing-agent malfunction can produce. The bound is upward-facing — the operator needs to know it before granting authority, the protocol layer needs to assume it for HITL load planning, the operating model needs to plan around it for cycle-cadence decisions. A substrate without a clear blast-radius bound forces the Governance Layer to assume worst-case harm, which collapses authorization toward the over-constrained failure mode of Paper Six §1.2.

Isolation primitives are the substrate's mechanism for setting the bound. Process isolation prevents one agent's failure from contaminating another's execution. Capability tokens prevent an agent from invoking capabilities its envelope did not authorize. Tool sandboxing prevents a tool's effects from reaching beyond the substrate-mediated path. Credential scoping prevents an executing agent's authorizations from outlasting the execution. Together, these primitives compose to a defensible upper bound on harm. Individually, none is sufficient.

The principle behind the composition is the principle of least authority,[51] which the operating-systems security literature has carried forward for fifty years. The application to agentic substrates is not novel as a security idea; what is novel is the substrate-design discipline of treating isolation primitives as governance contracts rather than as security

---

51 Jerome H. Saltzer and Michael D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE* 63, no. 9 (1975): 1278–1308, https://doi.org/10.1109/PROC.1975.9939.

features. Security features are conventionally evaluated by their robustness against attack; governance contracts are evaluated by their contribution to the audit-grade evidence chain. A substrate's isolation primitives must satisfy both evaluation criteria, and the substrate-research persona's dossier must score harness candidates on both.

5.2 PROCESS ISOLATION

The most basic isolation primitive is process isolation: each agent runs in its own operating-system process, with no shared memory across agents except through governed channels. The primitive is well-understood, widely available, and frequently absent from agentic deployments because it costs operational complexity that monolithic deployments avoid.

The governance argument for process isolation is direct. An agent that crashes, hangs, or executes a corrupting action affects only its own process; the orchestrator and the other executing agents continue. An agent whose memory is corrupted (by a tool return that did not match expected schema, by a context that grew past its bounds, by an input that triggered an error) does not contaminate other agents' memory. The blast radius of a single execution failure is bounded at the process boundary.

Process isolation extends to *container isolation* in containerized substrates: each agent runs in its own container, with filesystem isolation, network egress control, and capability restriction. Container-based isolation is heavier than process-based isolation but supports stronger guarantees: the agent's filesystem reads are bounded to the container's mount points, the agent's network reads and writes are bounded to the container's network policy, and the agent's syscalls are bounded by the container runtime's seccomp profile. For governance-grade deployments handling sensitive workloads, container isolation is the appropriate baseline; process isolation suffices for lower-risk classes.

The substrate's commitment is to provide process or container isolation by default, not as an opt-in. An agentic substrate that runs all agents in a single shared Python interpreter, sharing the global namespace, is a substrate whose blast-radius bound is "the entire substrate." That bound is too high for governance-grade deployment, regardless of how careful the application code is.

5.3 CAPABILITY TOKENS

Capability tokens are substrate-issued credentials that scope what an agent may invoke. The token concept comes from the operating-systems security literature[52] and applies cleanly to agentic substrates: the token authorizes specific capabilities (which tools, which APIs, which resources), is bounded in time (issued at admission, expired at execution close), and is non-transferable (an agent cannot give its token to another agent, and the substrate refuses tokens presented by an agent other than the issuee).

In an agentic substrate, capability tokens implement the capability-scoping requirement of §2.4. The admission hook (§4.2) issues a token when admitting a task; the token's scope reflects the envelope's authorized capabilities; the agent presents the token when invoking tools or resources; the substrate refuses calls whose tokens are missing, expired, or out-of-scope. The substrate is the single point at which token issuance, validation, and revocation happen, which makes the token a governance primitive rather than a convention.

---

52 Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano, "Passwords and the Evolution of Imperfect Authentication," *Communications of the ACM* 58, no. 7 (2015): 78–87, https://doi.org/10.1145/2699390.

Three properties make tokens governance-fit. First, *envelope-bounded scope*: the token authorizes what the envelope authorized, no more. A capability the envelope did not name is not on the token. Second, *time-bounded lifetime*: the token expires at execution close, not at session close. An execution that completes returns its token to the substrate; subsequent executions issue fresh tokens for fresh envelopes. Third, *audit-emitting issuance and use*: each token issuance and each token-bearing call is an evidence record on the channel of §4.3. The audit trail allows post-hoc reconstruction of which capabilities were authorized when and used by whom.

Most agentic substrates today do not use envelope-bounded capability tokens. Tools are made available through registry-level configuration; an agent that has been registered with the orchestrator can invoke any registered tool, subject to whatever conventional restrictions the orchestrator imposes. The pattern is operationally simple but governance-thin. Migration to capability-token-based access is a substrate-engineering project that the substrate-research persona's dossier should track per harness; some harnesses (notably MCP, with its capability negotiation primitives) have token-adjacent affordances that can be extended to envelope-bounded tokens with bounded effort.

5.4 TOOL SANDBOXING

Tools execute in isolated sandboxes. The sandbox bounds the effects of the tool's execution to a substrate-mediated path: the tool's filesystem reads are bounded to substrate-provided inputs, the tool's network access is bounded to substrate-mediated proxies, the tool's outputs are captured by the substrate before reaching the agent. The agent receives a tool result that has been mediated by the substrate; it does not receive whatever the tool produced.

Sandboxing is the mechanism by which tool calls become safe to admit. An agent that calls a filesystem-reading tool, in a substrate without sandboxing, has whatever filesystem access the substrate's process has — which is typically substantial in a development environment and unbounded in a misconfigured deployment. An agent that calls the same tool in a sandboxed substrate has access only to what the sandbox grants, which is typically a substrate-curated subset of the filesystem with read-only permissions and explicit path scoping.

The Model Context Protocol[53] is the most-deployed standard for sandboxed tool access in current agentic substrates. An MCP server sits between the agent and the underlying capability; the agent invokes the tool through the MCP protocol, the MCP server executes within its own sandbox, the MCP server returns a result. The sandboxing is at the MCP server boundary. The MCP architecture is governance-aligned in principle and increasingly so in practice, but governance-fit MCP deployment requires that the substrate gate the agent-to-MCP boundary as well: an agent that bypasses the substrate's MCP-call interception is acting on tools without admission, even if the MCP server itself sandboxes the tool execution.

Sandboxing must compose with the other isolation primitives. A sandboxed tool whose access is granted by an unbounded capability is sandboxed-but-overprivileged; the sandbox's restriction is undone by the capability's lack of restriction. Conversely, a tightly-scoped capability whose execution is unsandboxed is admission-controlled-but-effect-uncontained; the capability admits a narrow action that, once executed, can produce broad effects. Both failure modes are observable in current substrate landscapes; neither is necessary if the substrate composes the primitives correctly.

53 Anthropic, "Model Context Protocol Specification," 2024, https://modelcontextprotocol.io/.

5.5 CREDENTIAL MANAGEMENT

Credentials — API keys, OAuth tokens, database passwords, signing keys — are the means by which the substrate authorizes access to external systems. Credential management is the substrate-side discipline by which credentials reach the agents that need them, in the scope they need them, for the duration they need them, and not otherwise.

The governance requirement is that credentials are not embedded in agent memory. An agent that holds a credential in its working memory has, in effect, full authority over whatever the credential authorizes for as long as the agent's memory persists. The credential's scope is whatever the credential's issuer granted; the credential's lifetime is whatever the credential's issuer set. Both are typically broader than the envelope under which the agent is currently executing.

The substrate-mediated alternative is credential injection at admission. The substrate holds the credential. At admission, the substrate issues an envelope-scoped, time-bounded handle to the credential — not the credential itself. The agent presents the handle when invoking the system that requires authentication; the substrate intercepts the call, substitutes the actual credential, executes the call, and returns the result. The agent never sees the credential. The credential's effective scope is the envelope's scope; the credential's effective lifetime is the execution's lifetime. The substrate revokes the handle at execution close.

Credential injection is the right pattern for high-value credentials (production database access, signing keys, payment APIs). For lower-value credentials, credential-handle injection may be over-engineered, and direct injection of a scoped credential into the agent's environment may be acceptable — provided the credential is itself envelope-scoped and time-bounded at issuance. The principle is that credential scope and lifetime are bounded by the envelope, regardless of the implementation pattern.

5.6 THE BLAST-RADIUS THEOREM

A substrate that provides process isolation, envelope-bounded capability tokens, sandboxed tool execution, and envelope-scoped credentials supports a specific upper bound on harm. The bound is informally: the maximum damage from an admission failure is the union of capabilities the failed envelope authorized, executed within the time bound of the credentials.

Formally stated, the bound is the substrate's contribution to the operator's authority decision. The operator authorizing an envelope is, in effect, accepting the substrate's bound on the consequences of an admission failure under that envelope. If the bound is high (the envelope authorized broad capabilities, the credentials are long-lived, the sandboxing is loose), the operator's authority decision is consequential and should be subjected to additional review. If the bound is low (the envelope's capabilities are narrow, credentials are short-lived, sandboxing is tight), the operator can authorize with lower review overhead. The substrate's isolation primitives are what make the bound calculable.

The theorem is a contract, not an empirical claim. The substrate guarantees the bound; the operator authorizes against it. An admission failure that produces harm exceeding the bound is a substrate failure: either the isolation primitives were inadequately implemented, or the bound was incorrectly stated, or the failure was outside the substrate's scope (a vulnerability in the underlying operating system, a compromise of the substrate's own infrastructure). All three are diagnostic outcomes that the substrate-research persona should investigate; none is something the operator's authorization should have anticipated within the substrate's stated bound.

5.7 DEFENSE-IN-DEPTH POSTURE

No single isolation primitive is sufficient for governance-grade deployment. Process isolation without capability tokens leaves any agent with full access to the substrate's capability registry. Capability tokens without sandboxing produce tightly-scoped capabilities whose execution is uncontained. Sandboxing without credential management leaves credentials accessible to any sandbox-escape. Credential management without process isolation does not survive an agent that corrupts the substrate's process.

The substrate's commitment is defense-in-depth: all four primitives operate, and the failure of any one is contained by the others. Process isolation contains capability-token misuse to a single process. Capability tokens contain process-level escapes to envelope-authorized capabilities. Sandboxing contains tool-call effects to substrate-mediated paths. Credential management contains credential exposure to envelope-scoped handles. The composition is the bound; no individual primitive is the bound.

The defense-in-depth posture is the substrate's response to the recognition that capability finds paths. Each primitive closes some paths; the composition closes the paths each individually misses; the result is a substrate whose blast radius is small enough to authorize against. A substrate that ships all four primitives is governance-fit on the isolation surface; a substrate that ships fewer should be evaluated on which paths remain open and whether the deployment context can tolerate them.

---

6. SCALING THE SUBSTRATE ACROSS DEPLOYMENT TIERS

The substrate must function across a range of deployment scales. Four tiers cover most production agentic deployments: laptop (single developer or operator on a single machine), single-node server (a dedicated machine running the substrate), cluster (a Kubernetes or K3s cluster running the substrate as distributed services), and multi-region (substrate components distributed across geographic regions, typically for resilience or latency).

Each tier introduces new governance requirements that earlier tiers did not face. The laptop tier handles a small number of agents in a single process tree; manifest propagation is in-memory, partition isolation is at the process level, and operator interaction is direct. The single-node tier separates agents across multiple processes on the same machine; manifest propagation crosses process boundaries (though still in-memory or via local IPC), partition isolation may use namespace separation, and operator interaction begins to require remote-friendly affordances. The cluster tier separates agents across multiple machines; manifest propagation crosses network boundaries, partition isolation is at the namespace or cluster level, and operator interaction is unambiguously remote. The multi-region tier introduces network partitions, latency bounds, and consistency questions; manifest propagation must handle delays, partition isolation must handle cross-region drift, and operator interaction requires region-aware routing.

The governance contracts of §6.1 hold at every tier. What changes is the substrate-side mechanism by which they are honored. A substrate selected for the laptop tier without consideration of the cluster-tier requirements will discover, at the time of cluster deployment, that primitives the laptop substrate handled in-memory require redesign for distributed operation.

Two scaling dimensions cut across the four tiers. Some properties scale linearly with size: agent count, tool count, task throughput, evidence volume, manifest count. Linear scaling is well-understood — add capacity, get throughput. The substrate's job is to ensure that capacity

addition does not break the governance contracts. Other properties scale structurally with size: governance contracts. The contracts hold at every tier; the substrate primitives that enforce them must scale to match. Manifest propagation that worked in-memory at the laptop tier must work over the network at the cluster tier without losing integrity verification. Partition isolation that worked at the process level must work at the namespace level without losing the curation-history boundary. Drift detection that worked across local memory stores must work across cluster-distributed stores without losing the cross-context audit. The structural scaling is the design challenge; linear scaling is the operational challenge.

The scaling table summarizes which substrate primitives must be available at each tier, which can be approximated, and which break and require redesign.

Primitive	Laptop	Single-node	Cluster	Multi-region
Admission hook	In-process callback	IPC-mediated callback	Network-mediated, with sticky orchestrator	Region-local, with cross-region replication
Evidence channel	In-memory log			

Local file or DB
Cluster-shared store
Region-distributed, with eventual consistency
Manifest propagation
In-memory pass
Serialize on IPC
Serialize on network
Serialize with vector-clock or lamport ordering
HITL escalation
Direct operator interrupt
Local operator queue
Remote operator queue with notifications
Region-routed queue
Capability tokens
In-memory issuance
Local-secret-signed
Cluster-secret-signed
Region-coordinated signing
Process isolation
OS process
OS process
Container
Container, in regional pods
Sandboxing
Local sandbox
Local sandbox
Container-based sandbox
Region-isolated sandbox
Partition isolation
Process-level
Namespace-level
Namespace + cluster role
Region + namespace + cluster role

The table is illustrative, not exhaustive. The point is that each primitive has a tier-appropriate implementation, and a substrate-selection decision should consider the deployment tier the substrate must reach over its lifetime, not just the tier it begins at.

An operating model that runs on a laptop and on a cluster must produce identical evidence. The same envelope, admitted under the same manifest, executed by the same persona class, must produce evidence that the operator can audit identically regardless of which tier the execution happened on. The substrate's commitment is tier-invariant evidence semantics: serialization formats are fixed, manifest schemas are fixed, audit-channel field structures are fixed. What varies is the transport, the storage, and the latency; what does not vary is the semantic content. The cross-deployment consistency requirement has a practical implication: substrate selection should favor primitives whose semantics are tier-invariant. A substrate that uses different evidence formats at different tiers — JSON at the laptop, protocol buffers in the cluster, region-specific schemas at multi-region — produces evidence the operator must reconcile across tiers, which is reconciliation overhead the substrate should have absorbed. Tier-invariant primitives push the variability into the transport layer, where it belongs, and keep the governance-relevant content stable.

When two organizations' substrates collaborate (Paper Six §6.3), the cross-substrate consistency requirement is the same as the cross-tier requirement, with an added trust dimension: each organization's substrate must produce evidence the other organization's substrate can verify. The verification typically uses the same mechanisms as cross-tier audit (signed evidence records, manifest checksums, capability-token signatures); the additional discipline is that the signing and verification keys are organization-scoped rather than substrate-scoped. The Skipjack Protocol's future-study vector on cross-organizational interoperability[54] applies, and the substrate-side requirements are an extension of the cross-tier requirements rather than a separate design problem.

6.1 IMPLICATIONS FOR HARNESS-BASED IMPLEMENTATIONS

An agentic substrate implementation of this governance doctrine on top of a standard agent harness would require the following from the substrate:

- *Substrate-level admission interception.* The harness must expose a pre-execution hook that intercepts every agent action before it reaches a tool, an LLM, or another agent. The hook must be the only path: no alternative API, session-cached context, or back-channel call may bypass it. The governance flow requires this because admission discipline is the structural mechanism by which scope boundaries are enforced; without substrate-side interception, admission becomes an advisory check that capable agents will, over enough executions, route around. Failure mode if absent: tasks reach agents without scope manifests, AE² fails open, and the cycle's outputs cannot be audited back to a Governance-Layer authorization.
- *First-class evidence emission.* The harness must emit structured AE² evidence on a substrate-side channel distinct from agent stdout, tool returns, and diagnostic logging. Evidence records are write-only from the agent's perspective, append-only at the channel level, and per-task isolated. The governance flow requires this because the orchestrator's completion review and the operator's audit depend on substrate-witnessed evidence rather than self-reported claims; an agent that constructs evidence about its own behavior is providing narrative, not audit. Failure mode if absent: the AE²

54 Kuiper, "Skipjack Protocol," §7 Vector 5 (Cross-organizational Skipjack interoperability).

return becomes a story the agent tells about its own actions, and the cycle's compliance cannot be reconstructed from durable artifacts.

- *Manifest persistence and propagation.* The harness must serialize, transport, and deserialize the Skipjack context integrity manifest across every handoff in the execution chain. Serialization must be deterministic, deserialization must verify integrity, and each node's handoff must emit a receipt. Agents may read the manifest but cannot mutate it; the manifest the next node receives is the manifest the substrate transported. The governance flow requires this because context integrity across handoffs is what makes the chain-of-custody trail meaningful; a manifest that agents can modify becomes a self-asserted summary, not a governance artifact. Failure mode if absent: cross-handoff context drift, with later nodes acting on a manifest other than the one the Governance Layer authorized.
- *Capability-scoped credential issuance.* The harness must issue credentials at admission time, scoped by the envelope, with an enforced time bound. Credentials may be substituted by the substrate at the point of external-system call (credential-handle injection) or scoped at issuance (envelope-bounded credentials), but their effective scope must be the envelope's scope and their effective lifetime must be the execution's lifetime. The governance flow requires this because an agent holding broader-than-envelope credentials has broader-than-envelope authority for as long as the credentials persist, which defeats admission discipline retroactively. Failure mode if absent: agents accumulate latent authority across executions, and the blast-radius bound the operator authorized against is unenforceable.
- *Substrate-side partition isolation.* Multi-context partitions must be structural — process boundaries, namespace separation, cluster-role boundaries — not prompt-side personas on a shared underlying context. Partition isolation must hold against substrate-internal cross-references; an agent in one partition cannot read another partition's curation history except through governed re-unification points. The governance flow requires this because the operating model's multi-account discipline (Paper 6 §4) depends on context-isolation as governance, not as aesthetic separation; saturation contaminates discrimination regardless of how careful the prompts are. Failure mode if absent: cognitive-mode contamination, indiscriminable curation history, and gradual collapse toward single-persona theater.

These are governance contracts that any compliant implementation must honor, regardless of harness substrate.

The contracts are unprecedented to varying degrees in current harness frameworks. Substrate-level admission interception is the most-precedented; CrewAI's process manager,[55] LangGraph's[56] node functions, MetaGPT's[57] SOP-driven orchestration, and the OpenAI Agents

55 CrewAI, "CrewAI Documentation," 2024–2026, https://docs.crewai.com/. (No peer-reviewed publication as of this writing.)

56 LangChain, "LangGraph Documentation," 2024–2026, https://langchain-ai.github.io/langgraph/. (No peer-reviewed publication as of this writing.)

57 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024, https://arxiv.org/abs/2308.00352.

SDK's[58] tool-call interception each provide adjacent primitives that can be extended to envelope-bounded admission with bounded engineering effort. First-class evidence emission is novel in agentic substrates; most harnesses log to stdout or to harness-internal stores not designed for governance audit. Manifest persistence and propagation is novel — no standard harness ships the Skipjack context integrity manifest as a substrate primitive — and is the most substrate-engineering-heavy contract. Capability-scoped credential issuance has precedent in MCP's[59] capability negotiation but not as envelope-bounded scoping. Substrate-side partition isolation is generally available across separate harness instances but not as substrate-internal partitioning with shared-state guarantees.

The implication for substrate selection is that no current harness satisfies all five contracts out of the box. Compliant implementations will compose harness affordances with substrate-engineering work. The doctrine is teachable; the substrate-side implementation is engineering, and the substrate-research persona's dossier is the artifact under which that engineering should be tracked across deployments.

---

7. FUTURE STUDY: TEN RESEARCH VECTORS

The agentic-substrate specification in this paper is governance doctrine, not a validated implementation. The ten research directions below identify where the doctrine requires empirical grounding, where its claims need formal verification, and where the operative context will likely force extension.

1. Empirical evaluation of harness-substrate fitness against the five §6.1 contracts. The five governance contracts of §6.1 are stated as substrate-independent requirements. Whether a given harness — LangGraph, AutoGen, CrewAI, MCP, OpenAI Agents SDK, MetaGPT, ChatDev — can be made to satisfy each contract, with what engineering effort, and at what runtime cost, is an empirical question. The research design is a benchmark suite that evaluates a harness against each contract under representative task loads. The substrate-research persona's dossier is the natural locus for accumulated findings; the research challenge is constructing benchmarks that test governance enforcement (the gate fires when it should and does not when it should not) rather than capability throughput.

2. Formal specification of substrate primitives as machine-verifiable interfaces. The admission hook (§4.2), evidence channel (§4.3), and manifest propagation (§4.4) are described in natural language. A formal specification — type-theoretic, interface-description-language-based, or process-calculus-based — would allow harnesses to attest, at deployment time, that their implementations satisfy the substrate contract. The Skipjack §7 vector on formal HITL specification[60] is closely related; substrate-primitive specification is its substrate-layer counterpart. The expressiveness-vs-tractability tradeoff applies: a specification rich enough to capture the contracts may be too rich for tractable verification.

3. Substrate composition — when multiple harnesses compose in one operating model. Few production deployments use a single harness exclusively. A typical composition

---

58 OpenAI, "OpenAI Agents SDK Documentation," 2024–2026, https://platform.openai.com/docs/. (No peer-reviewed publication as of this writing.)

59 Anthropic, "Model Context Protocol Specification," 2024, https://modelcontextprotocol.io/.

60 Kuiper, "Skipjack Protocol," §7 Vector 1 (Formal specification of HITL control point topologies).

uses LangGraph for orchestration, MCP for tool access, an OpenAI Agents SDK for some agent classes, and bespoke code for substrate-specific primitives. Which harness carries which contract — orchestration through LangGraph, manifest propagation through bespoke code, capability scoping through MCP — is a composition design question. The research question is whether substrate composition can be expressed as a contract-by-contract assignment and whether the assignment can be machine-verified before deployment.

4. Inference-substrate independence — the open-source-only posture rationale at scale. Paper 5 §10 stated the open-source / shared-pool / locally-hosted posture; this paper operationalized it in §4.7. The posture's scaling properties are not yet established. Specifically: at what deployment scale does locally-hosted inference cost more than the governance benefit it delivers? The research question is empirical: a deployment-cost study that compares locally-hosted inference (with full governance integration) against proprietary-API inference (with governance-as-convention) across task classes, throughput levels, and operator overhead. The answer is likely deployment-context-dependent; the research goal is to characterize the dependence so deployments can decide informedly.

5. Capability-token revocation and audit. Capability tokens (§5.3) authorize tool access at admission. Revocation — pulling a token before its time-bound expiry — is necessary in scenarios where mid-execution events change the authorization state (an operator escalates, a drift report surfaces a contradiction, an SOP propagation requires acknowledgement). The substrate-side mechanics of revocation, the latency between revocation and effect, the audit of revocation events, and the recovery of executions interrupted by revocation are all open. The classical operating-systems literature on capability revocation provides starting points; agentic-substrate adaptation has not been done.

6. Substrate change governance — substrate swap as a Governance-Layer-approved migration. Substrate swaps happen: a deployment that began on one harness migrates to another as the harness landscape evolves. The research question is whether substrate swaps should be treated as Governance-Layer-approved migrations (with a documented governance-fit assessment, an evidence-chain continuity proof, and a rollback path) or as runtime configuration changes (the orchestrator transparently swaps the substrate, and the operating model continues). The argument for the migration treatment is that a substrate's governance properties are part of the deployment's audit-grade evidence chain; a swap that does not preserve those properties breaks the chain in a way the operator must adjudicate. The substrate-research persona's dossier should track this question across observed swap events.

7. Substrate observability standards — telemetry sufficient for Skipjack manifest verification. The Skipjack context integrity manifest carries a chain-of-custody trail. The substrate-side telemetry sufficient to populate the trail — handoff receipts, integrity verifications, transport logs — has not been standardized. A substrate-observability standard would allow operators to verify, against a published schema, that a candidate substrate produces telemetry the manifest can use. The OpenTelemetry community has adjacent prior art; the agentic-substrate-specific extensions are open.

8. Transitive substrate dependencies — model providers, vector stores, tool integrations. A substrate composes more than the harness. Model providers (whether locally hosted or proprietary), vector stores (for episodic memory), tool integrations (each external system the agent can call), and supporting infrastructure (observability stacks, secrets managers, queue systems) all participate in the substrate's governance flow. The research question is whether the governance contracts of §6.1 propagate transitively: when a substrate depends on a vector store, must the vector store satisfy substrate-grade isolation, audit, and credential discipline? The general answer is yes; the operational question is how the

propagation is enforced when transitive dependencies are themselves third-party systems with their own governance properties.

9. Substrate hardening for adversarial agents. This paper has assumed agents that are non-adversarial — capable, sometimes drift-prone, but not deliberately attempting to defeat the substrate. The adversarial case (an agent that has been compromised, an agent whose underlying model has been adversarially trained, an agent fed adversarial inputs that produce capability-extension behavior) is increasingly important as agentic deployments expand. The research question is which substrate primitives still hold under adversarial agents and which require strengthening. The classical security literature on adversarial-trust execution applies; the agentic-substrate specifics — adversarial reasoning loops, adversarial tool selection, adversarial manifest manipulation attempts — are open.

10. Substrate evolution and the operating-model cycle. Substrate updates happen: harness versions advance, isolation primitives improve, governance affordances extend. Coordinating substrate updates with operating-model versioning (Paper 6 §7 vector 10) is the cross-paper question. A substrate update that deprecates a primitive the operating model depends on is a coordination event the protocol layer must adjudicate. The research question is the version-management discipline: how substrate versions are declared, how operating-model versions express substrate dependencies, how migrations are sequenced. The deployment-engineering literature on database migrations and API versioning provides starting points; agentic-substrate adaptation has not been done.

---

8. CONCLUSION

The agentic substrate described in this paper is the layer at which capability becomes governable. The Framework specifies the doctrine, the Protocol specifies the authority structure, the Operating Model specifies the cadence — and the Substrate is what physically permits any of them to be expressed. A substrate that does not surface admission as a refusal point cannot host the Skipjack admission gate. A substrate that does not propagate the context integrity manifest across handoffs cannot host the Framework's C³ contract. A substrate that does not partition cognitive modes structurally cannot host the operating model's multi-account discipline. The substrate is not a place where governance is added; it is the place where governance is either expressible or not.

The four claims of the substrate specification have been defended through the paper. *Agent anatomy is a governance surface, not a capability surface*: memory must carry curation lineage, reasoning must consult the admission gate, tool access must be capability-bounded. *Orchestration topology determines authority distribution*: single-agent collapses orchestrator into executor; multi-agent separates them but requires substrate-side role typing; swarm distributes authority and requires consensus mechanisms the protocol layer must specify. *Governance integration is substrate-layer, not protocol-layer*: protocol-layer specifications are honored by agents that choose to honor them; substrate-layer interceptions are encountered by agents that cannot route around them. *Runtime isolation is the blast-radius contract*: process isolation, capability tokens, sandboxing, and credential management compose into a defensible upper bound on harm.

The five governance contracts of §6.1 are the substrate's deliverable: substrate-level admission interception, first-class evidence emission, manifest persistence and propagation, capability-scoped credential issuance, and substrate-side partition isolation. No current harness satisfies all five out of the box. Compliant implementations compose harness affordances with substrate-engineering work, and the substrate-research persona's dossier (yks-build #73 / yks-

spine-binder #227) is where that work is tracked across the harness landscape. The doctrine is the contracts; the implementation is the engineering; the dossier is the bookkeeping.

The portfolio's working substrate — the yks-llm-gateway on K3s, the four shared model pools (command, code, narrative, analysis), the substrate-research persona's gateway operations — is an existence proof at small scale. The decalogy was written under it. The papers ship at v1.0-preprint with audit-grade evidence chains, the cycle's decisions are ratified through completion reviews, the SOP propagation log gates tasks where acknowledgements are owed, and the drift reports surface (and resolve) the small divergences each cycle inevitably produces. This paper specifies the substrate that the paper's own production demonstrates is implementable. The proof is in the working portfolio, not in the prose.

The substrate is not finished work. The ten research vectors of §7 identify where empirical grounding is needed, where formal specification would harden the doctrine, and where the operating context will likely force extension. The capstone implementation paper (Paper 9) and the capstone results paper (Paper 10) will produce the empirical evidence. This paper's job is to specify the substrate in a form precise enough that the engineering can be built against it.

The agentic substrate is the layer at which capability becomes governable — not by adding governance on top of capability, but by selecting substrate primitives whose shape forces governance to be expressible in the first place.
